

CMAN — Roofline Analysis for the QK^T Accelerator

ECE 410/510 • Hardware for AI and ML • Codefest 09

Pratibha M

Overview

This writeup applies the roofline model to my QK^T accelerator as it actually came out of synthesis in M3 not the M1 projection. The M1 numbers were a sizing target written before I understood what sky130 would let me close timing on, and a lot has changed since then. MS3 is a 4x4 systolic array of FP16-multiply / FP32-accumulate PEs, closing at 33.3 MHz on the typical PVT corner with +9.06 ns slack.

I want the roofline to reflect that hardware, not the slide-deck version.

I work through the five tasks in order. The two AI bounds in Task 3 are the headline of this codefest — they bracket where my design can sit on the roofline depending on how well the on-chip buffering captures GEMM reuse. The plot then overlays three platforms: the M1 software baseline (i5-1235U laptop), the M3 synthesized hardware, and an M4 projection assuming the FP32-adder pipelining work in my M4 backlog actually lands. Task 5 calls out the bottleneck and what I plan to do about it before final submission.

Task 1 — Dominant kernel

The kernel is the QK^T scaled dot-product score computation in transformer self-attention. From my M1 cProfile run, it consumed 91% of NumPy runtime, which is why I picked it as the offload target in the first place. Mathematically:

$$\text{scores}[b, h, i, j] = (1/\sqrt{d_{\text{head}}}) \cdot \sum_k Q[b, h, i, k] \cdot K[b, h, j, k]$$

It's a batched, multi-head dense GEMM. I'm benchmarking at the same operating point as my M1 software baseline (B=8, H=4, T=64, d_{head} =16) so the hardware and software numbers stay directly comparable.

Parameter	Value
Compute engine	4x4 output-stationary systolic array (16 PEs)
PE arithmetic	FP16 multiplier + FP32 accumulator
Closed clock (nom_tt)	33.3 MHz (30 ns period, +9.06 ns slack)
External interface	128-bit AXI4-Stream @ 33.3 MHz
Process	sky130_fd_sc_hd, OpenLane 2 (GDS-clean signoff)
Area	315,688 μm^2 (~0.32 mm ²)

Power @ nom_tt	162.2 mW (OpenSTA estimate)
----------------	-----------------------------

One thing worth flagging before the AI calculations: the M1 baseline ran in NumPy FP64 (8 bytes/element), but the hardware moves FP32 (4 bytes/element on the accumulator path). Since arithmetic intensity divides FLOPs by bytes, the same kernel ends up with different AI numbers on the two platforms. I compute both below and label them, rather than picking one and creating an inconsistency.

Task 2 — FLOPs per kernel invocation

Standard dense-GEMM count: each output element costs $\sim 2 \cdot d_head$ FLOPs (d_head multiplies, d_head-1 adds, rounded). Plugging in the M1 operating-point numbers:

$$\text{FLOPs} = 2 \cdot B \cdot H \cdot T^2 \cdot d_head = 2 \cdot 8 \cdot 4 \cdot 64^2 \cdot 16 = 4,194,304$$

So ~ 4.19 MFLOPs per call, or 2,097,152 MACs. This number is datatype-independent — it doesn't change between FP32 and FP64.

Task 3 — Bytes transferred and arithmetic intensity

QK^T has the classic GEMM operand-reuse pattern: each element of Q gets multiplied against every column of K^T ($T=64$ reuses), each element of K likewise gets reused across every row of Q , and each output element accumulates $d_head=16$ partial products before being written. That's the structural reuse the accelerator is meant to capture; how much of it actually gets captured determines whether I sit at the upper bound or the lower bound below.

Element counts (these are fixed by the kernel math):

Tensor	Shape	Elements
Q	(B, H, T, d_head)	32,768
K	(B, H, T, d_head)	32,768
scores	(B, H, T, T)	131,072
Total operands	—	196,608
MACs performed	—	2,097,152

Upper-bound AI — perfect on-chip reuse

This is the GEMM ideal: load each tensor from off-chip exactly once, hold it in on-chip SRAM, do all the math against the resident data, write the output once. The pattern name is GEMM operand reuse (with the output stationary in the accumulator). Formula:

$$\text{Bytes_perfect} = (|Q| + |K| + |\text{scores}|) \times \text{bytes_per_elem}$$

Datatype	Bytes/elem	Total bytes	AI = FLOPs/Bytes
FP64 (M1 software)	8	1,572,864 (1.50 MB)	2.667 FLOP/B

FP32 (M3 hardware)	4	786,432 (0.75 MB)	5.333 FLOP/B
--------------------	---	-------------------	--------------

Quick cross-check: the FP64 number lines up with what I got in CF02 (2.667 FLOP/B), and halving the byte width should exactly double the AI, which it does. Good.

Lower-bound AI — no data reuse

The worst case is a design with no on-chip buffering at all: every MAC re-fetches both operands from off-chip memory, and the output is written once at the end. Formula:

$$\text{Bytes_no-reuse} = (2 \cdot \text{MACs} + |\text{scores}|) \times \text{bytes_per_elem}$$

Datatype	Bytes/elem	Total bytes	AI = FLOPs/Bytes
FP64	8	34,603,008 (33.0 MB)	0.1212 FLOP/B
FP32	4	17,301,504 (16.5 MB)	0.2424 FLOP/B

Cross-check again: the lower bound should asymptote toward 2 FLOPs per (2 · bytes_per_elem) of operand traffic, i.e. 0.125 for FP64 and 0.25 for FP32. The small offset below those numbers comes from the once-only output writes — also makes sense.

What the two bounds mean together

The spread between the lower and upper bound is 22× — same factor for both datatypes, since it's a property of the kernel, not the bit-width. That factor is the whole reason on-chip buffering matters. Without reuse, this kernel is deeply memory-bound on any plausible platform. With perfect reuse, it's compute-bound on any plausible platform. Where you actually land depends on the tiling and SRAM strategy of the accelerator, and I'll come back to that in Task 5 because my current M3 reuse pattern is not at the upper bound.

Task 4 — Roofline construction

I'm plotting three rooflines on the same axes, so the comparisons land in one figure. Each comes from a different source:

- M1 software baseline (Intel i5-1235U laptop): peak compute 176 GFLOP/s (AVX2 across 2 P-cores + 8 E-cores in FP64), peak BW 76.8 GB/s (LPDDR5-5200, dual-channel). Ridge sits at 2.29 FLOP/B. These numbers are from my M1 sw_baseline.md and they're the i5-1235U datasheet values.
- M3 hardware (sky130, as synthesized): peak compute = 16 PEs × 2 FLOPs/cycle × 33.3 MHz = 1.067 GFLOPS. Peak BW = 128 bits × 33.3 MHz / 8 = 0.533 GB/s. Ridge = 2.00 FLOP/B. These come straight out of my OpenLane signoff.
- M4 projected (sky130 + adder pipelining): same array, but with the FP32-adder cut into pipeline stages so the clock can move up to ~100 MHz (the target from my critical_path.md). Peak compute becomes 3.2 GFLOPS, peak BW becomes 1.6 GB/s. Ridge is still 2.00 FLOP/B — clock scaling lifts the roofline diagonally and doesn't change where the ridge sits.

That last point is worth pausing on. When you raise fmax, the compute ceiling and the BW ceiling go up in lockstep (because the AXI interface is clocked at the same rate as the compute fabric), so the ridge

point stays put. To actually shift the ridge you'd have to widen the AXI bus or scale the array independently of the clock. I don't plan to do either of those before M4.

Where the kernel sits on each roofline

Using each platform's relevant datatype AI (FP64 for software, FP32 for hardware):

Platform	AI (upper)	Ridge	Attainable
M1 software (FP64)	2.67	2.29	176 GFLOP/s (compute)
M3 (FP32)	5.33	2.00	1.07 GFLOPS (compute)
M4 projected (FP32)	5.33	2.00	3.20 GFLOPS (compute)

All three are compute-bound at the upper-bound AI. That's expected for dense GEMM. The interesting thing the roofline reveals is the no-reuse case: at AI = 0.24 FLOP/B, the M3 design only achieves 0.13 GFLOPS — a dramatic collapse from the 1.07 GFLOPS compute ceiling, and a reminder that the on-chip buffering strategy is doing all the heavy lifting.

The roofline sketch is saved as `cman_roofline_sketch.pdf` (and `.png`) in the same folder as this document. It shows all three rooflines, both AI bounds as vertical guides, the M3 attainable performance plotted at both ends of the AI range, and the M1 measured throughput marked on its own roofline for context. The M4 line is drawn dashed and faded to keep the projection visually distinct from the synthesized facts.

Task 5 — Bottleneck and highest-leverage change

Where is the design limited right now?

The M3 design is compute-bound at the perfect-reuse upper-bound AI of 5.33 FLOP/B. That AI is well right of the 2.00 FLOP/B ridge, so the binding ceiling is the compute roof at 1.07 GFLOPS, not the AXI interface and not on-chip SRAM. (My internal Q/K-buffer bandwidth is in the hundreds of GB/s range — nowhere near the binding constraint.)

The physical reason the compute ceiling is only 1.07 GFLOPS is documented in my `critical_path.md`: the FP32 adder has roughly 58 levels of combinational logic post tech-mapping, dominated by two variable barrel shifters (24-bit right-shift to align mantissas before the add, 24-bit left-shift to normalize after). That's the structurally deepest part of any IEEE-754 floating-point add, and it's what forces the 30 ns clock period. At the slow-slow PVT corner the same path takes about 41 ns and the design fails setup by ~12 ns, which confirms the adder really is the binding piece of silicon (the SS-corner closure is in my M4 backlog).

One caveat: ceiling vs measured utilization

The above is a roofline-ceiling argument. It says where I could be at 100% utilization. A separate question is what fraction of that ceiling M3 actually reaches in simulation, and I haven't measured this carefully yet. `tb_top` end-to-end runtime is 9,255 ns for a single 16×16 invocation. A back-of-envelope compute-only floor is ~80 cycles × 30 ns = 2,400 ns of useful array work, which means the remaining

~6,800 ns is some mix of AXI4-Lite control writes, AXI4-Stream load/drain phases, FSM state transitions, and possibly testbench-host overhead that wouldn't exist in a deployed system.

I haven't broken this down from waveforms yet, so the roofline above reflects peak attainable, not measured utilization. If a large share of that 6,800 ns turns out to be real accelerator stall time (engine idle waiting on AXI loads), the highest-leverage M4 change might shift toward stream-FIFO or double-buffer improvements rather than just adder pipelining. Both changes help, but the priority order depends on which one is actually leaving more performance on the table. Measuring that breakdown is itself an M4 task, and the result will feed into the M4 design-justification report's priority ordering.

Highest-leverage single change

Subject to that caveat, the change I would single out is pipelining the FP32 adder internally. This is the P-top item in my M4 backlog. Concretely: insert two pipeline registers inside `fp32_adder.sv` — one after the exponent-compare and right-shift alignment, another after the mantissa add and left-shift normalization. That splits the 58-level path into three roughly balanced stages of ~20 levels each, which should close timing at 100–125 MHz on the typical corner. The cost is ~1024 additional flops across the array (64 new flops per PE × 16 PEs) and a +2 bump in `ARRAY_LAT` in `top.sv` to account for the extra latency.

Why this one first: it attacks the named binding constraint from a real post-PnR STA report, the effort is bounded (one RTL file, known testbench, known reference), and the payoff is the multiplicative one. Going from 33.3 MHz to ~100 MHz takes peak compute from 1.07 GFLOPS to ~3.2 GFLOPS, which is enough to bring the chiplet into roughly 75% parity with the M1 software measured throughput (4.16 GFLOP/s on the i5-1235U) at 0.32 mm² and 162 mW. That per-Watt and per-mm² story is the actual project narrative.

A pairing I'd add as a second change: restore full d-dim accumulation (the P-acc backlog item).

M3 scope-reduced the inner-dim window to D=4 per tile, which means Q and K get re-streamed across multiple invocations to compose a full 16×16 result. That breaks operand reuse and pushes the effective AI below the 5.33 upper bound, somewhere closer to the ridge. Without this fix, the adder-pipelining unlock gets partially eaten by extra AXI traffic. With both fixes together, the M4 design ends up at the projected point on the roofline, compute-bound, with the buffering actually doing the work it was designed to do.

What I'm not prioritizing for M4

A few things are tempting but probably wrong calls at this stage. Rewriting the 24-bit mantissa adder with carry-select would save 5–10 levels but the barrel shifters dominate, so the ROI is modest — noted as M5+ in my `critical_path.md` (which I'll have to fold into a “future work” section since the course finishes at M4). Scaling the array from 4×4 to 8×8 would multiply compute but push the ridge to 8 FLOP/B, crossing into the memory-bound regime at AI = 5.33 — not useful without also widening AXI,

and that's more than I can verify cleanly in the time I have left. Dropping accumulator precision to FP16 or BF16 would halve the barrel-shift depth and probably give the cleanest fmax win of all the options here, but it introduces a numerical deviation from the M1 reference and would require re-verifying tb_top against a new bit-exact reference. I'll mention this as a precision tradeoff in the M4 "what did not work / future variants" section, but it's not on the M4 critical path.